AutoBayes brms-parity loop: Tier G review

Status at the Tier G boundary, prepared for Andrew

3 November 2026

This is the Tier G boundary review. Since 26 October you asked for four things: continue with Tier G, releasing the two withheld results and three of F09's call-surface changes (with `target_accept` kept at 0.9); scale AutoBayes to larger models; make the speed comparison fair with cmdstanr and brms's threading; and pause at 90% of the weekly usage. Every Tier G feature but one is on `main`. The exception, G07 (the family–context matrix), was probed and is waiting on you, because it turned out to be about seventeen features rather than one. Section 2 is the scaling work, Section 3 the first fair benchmark, Section 4 what brms gets wrong, and Section 9 what needs you.

1 What was built

Table 1 lists the features. Each refuses, naming the construct and its divergence row, every combination no frozen case verifies. One fix merged on its own: **per-category predictions (#119)**. G11's agent noticed that a fitted dirichlet or logistic_normal model with `(1 | g)` could not be used for prediction: a fit's posterior holds the group effects as (category, level), the graph as (level, category), and `predict()` read them by position. It crashed when the levels outnumbered the non-reference categories or were fewer, and when they were equal it returned wrong numbers without an error (off by up to 0.47). `log_lik`, `waic` and `loo` were affected too. The oracle gates missed it because their draws are laid out in the graph's order. It now reads by dimension name.

How close. I measured with the gates' own comparisons on `main` after G10 (`92ab9b4`), leaving out only the gates each case is expected to fail (Section 7). The worst relative errors over the 69 Tier G cases that have gate tests:

- 2.2×10^{-16} in G1;
- 2.4×10^{-10} in G2's log density (`gauss_gp_matern32_default_prior`) and 1.7×10^{-9} in its gradient (`student_nu_formula`), a fortieth and a sixtieth of the tolerances;
- 1.9×10^{-13} in G3 and 3.1×10^{-13} in predictions (both `gauss_gp_matern32`);
- 8.0×10^{-15} between the JAX and PyTensor densities, and 2.9×10^{-15} in the post-processing tables.

Over all 285 cases the worst figures are still Tier F's (1.9×10^{-9} and 1.2×10^{-8} in G2, the second brms's own error, D150). Everything is inside the unchanged tolerances. The full non-sampling suite on `main` after G10 (`92ab9b4`) passed: 8,362 tests passed and none failed, with 271 skipped and 30 expected failures (2 h 3 min). `oracle/regen.py check -all` regenerated all 287 frozen fixtures from R on the same commit, and every one was identical.

Feature	PR	What a user can now write	Diverg.
G01 releases	#110	`trunc(lb = 1)` counts with the exact mean $E[Y \mid Y \geq lb]$; exact stacking weights, `ab.loo_model_weights`	D121, D126, D176
G02 call surface	#113	an undeclared pandas `Categorical` or bool column; `b={"*": ab.Normal(0, 2), "x": ...}`; `dropna=True`	D172–D174, D177–D178
G03 term algebra	#112	`x * z * h`, `(x + w + z)^2`, `x:w + z:h`, `z + x:z`, `x - w`	D179–D180
G04 transforms	#111	`I(x * w)`, `log(xp + 1)`, `scale(x) + poly(w, 2)`, `factor(o)`, several offsets	D181–D182
G05a group syntax	#114	`(0 + x | g)`, `(1 | g) + (0 + x | g)`, `(1 + z | g)`, `(1 + mo(o) | g)`, `(1 | g/h/k)`	D183–D184
G05b gr(), mm()	#115	`gr(g, cov = A)`, `gr(g, dist = "student")`, `(1 + x | mm(g, g2))`, `mmc()`	D185–D186
G06 distributional groups	#116	`sigma ~ z + (1 | g)`, `(1 |p| g)` across mu and sigma, `nu ~ z`, `zi ~ z + (1 | g)`, `disc ~ z`	D187–D188
G08 multivariate	#120	`bf(y ~ x) + bf(y2 ~ z)` with rescor; `mvbind(y, y2) ~ x + (1 |p| g)`; `mvbind(y, y2) | weights(w) ~ x`	D193–D194
G09 gp()	#121	`gp(x, z)`, `gp(x, z, iso = FALSE)`, `gp(x, by = f)`, `cov = "matern32"`, `scale = FALSE`	D195–D196
G10 special terms	#122	`mo(o):x`, `mo(o) * mo(o2)`, `mo(o, id = "a")`, `me(x, sdx, gr = g)`, `cs(x + z)`, `(cs(1) | g)`	D197–D198
G11 scale	#117	the same models at $N = 10^6$ rows and 10^4 groups (Section 2)	D199

Table 1: The Tier G features, verified on 70 new frozen brms cases: #108, #109 and #118 froze 68 before the features began, and G09 froze 2 more to check its approximate kernels.

2 Scaling to large models

Before G11, a logistic model `y ~ x1 + x2 + (1 + x1 | g)` could not be built at $N = 10^5$ rows and $J = 10^3$ groups: it needed about 60 GB. Most of that was not the sampler's graph, which already indexed by group code, but three dense $N \times J$ copies built around it:

- **the target identity**, which canonicalised every entry of the group design, about 290 bytes each;
- **formulae's indicator matrix** for each grouping factor, which Bambi keeps alive;
- **AutoBayes's own dense design**, plus `predict()`'s per-row Python loop.

G11 builds each group coefficient from integer level codes and hands it to Bambi's backend directly (your option A), streams the identity's bytes into the same digest (your choice), keeps the dense design only as a capped, lazily built view, and predicts by one gather per coefficient.

- **Same models.** Before changing anything, the agent recorded a manifest on `main`: for every oracle case, the identity, `describe()`, names, graph, design hashes (sign bits included), log density and gradient on PyTensor and JAX at the frozen points, and predictions; for every model the suite builds, the cheap fields. Afterwards 259 cases and 4,347 suite models were bit-identical, except 89 `kind="mean"` arrays that moved by at most 1.35×10^{-12} relative (summation order; you accepted that). `kind="response"` draws now come from AutoBayes's generator rather than Bambi's (D199).
- **Size.** The same logistic model at $N = 10^6$, $J = 10^4$ now builds in 204 s at 0.81 GB, and one log density with its gradient costs 24 ms in PyTensor and 8 ms in JAX. A test builds $N = 2 \times 10^5$, $J = 2 \times 10^3$ under 0.26 GB on every suite run.
- **What is left.** 197 of those 204 s are the identity's hashing: keeping the old digest means still

reading every entry of the $N \times \sum Jk$ design (about 2.3 GB/s). A new digest format over the codes would make it negligible but change every existing group model's digest (Section 9). Non-linear and count-matrix models keep dense designs, and `gr(cov = A)` keeps its $J \times J$ matrix.

3 A fair speed comparison

The benchmark now runs on FASRC (`sapphire` nodes, Xeon Platinum 8480+, one CPU model for every task) with brms 2.23.0 on rstan 2.32.7 and on cmdstanr 0.9.0 with CmdStan 2.40.0, compiled with `STAN_THREADS`. brms's configurations are rstan; cmdstanr without threads; `threading(t)` with $t = 1, 2, 4$; and at $t = 4$ also `static = TRUE`, and JK's `threading(4, grainsize = 250, static = TRUE)`. AutoBayes runs on 4, 8 and 16 cores under an enforced CPU affinity, so both libraries get the same cores. The pilot ran the five scaling models at 10,000 rows, one seed, 4 chains of 1000 warm-up and 1000 kept draws, target acceptance 0.9: 50 tasks, about 51 core-hours.

model	brms best	ESS/s	AutoBayes best	ratio	end to end
S1 logistic, slope	4 threads, static	41	26 (16 cores)	0.64	3.5
S2 Gaussian, 3 slopes	4 threads	6	14 (16 cores)	2.2	6.1
S3 Poisson, crossed	4 threads, static	9	34 (4 cores)	3.7	8.5
S4 ordinal	JK's (grainsize 250)	4	9 (8 cores)	2.0	4.5
S5 approximate GP	2 threads	1	2 (16 cores)	3.4	3.7

Table 2: Best configuration against best, bulk ESS per second of the samplers' time (the minimum over parameters); brms's configurations are cmdstanr's; ratio is AutoBayes's over brms's, of the samplers' time and end to end, which includes brms's 90–130 s compilation.

- **brms's threading works; AutoBayes's extra cores do not.** brms at 4 threads keeps 13 to 15 of its 16 cores busy. AutoBayes keeps about 3.85 busy at any cap: one per chain, with no parallelism inside a chain at this size. Given 16 cores, brms wins S1 (0.64); at 16 cores AutoBayes on S4 was even slower per gradient than at 8.
- **At equal cores AutoBayes is well ahead:** with 4 cores each (cmdstanr without threads), 3.3 times the bulk ESS per second on S1, 6.1 on S2, 12 on S3, 4.9 on S4 and 3.5 on S5.
- **JK's setting** was brms's best only on S4; on S1–S3 brms's default grainsize (1,250 here) did better. This may change at 10^5 and 10^6 rows.
- **S5 is a poor speed test as specified.** Both libraries take about 4 million leapfrog steps and report divergences; its priors or data need fixing before the full grid.
- **Limits.** One seed, the smallest size. The 100k and 1M grid waits for your decision (Section 9). The report is `scaling-2026-11-03-pilot.pdf` on the `bench/scaling` branch.

4 What brms gets wrong

The upstream log now has 54 entries. Tier G added two, each with a standalone script that exits non-zero while the problem is present:

- **U53, `gr(g, cov = A)` with several correlated coefficients.** brms's generated code skips every negative entry of $\mathrm{chol}(A)$, so the effects have covariance $L_+ L_+^\top \otimes \Sigma$ rather than $A \otimes \Sigma$ (an entry of 0.25 where A has 0.10). This was already reported as brms issue #1939 and is fixed in the development version (2.23.2), but CRAN still ships 2.23.0. AutoBayes builds the Kronecker product (D185).

- **U54, an ID shared between mu and sigma.** With an `sd` prior by group and another by `dpar = "sigma"`, `prior_summary()` gives sigma's SD normal(0, 0.5), while the Stan code applies normal(0, 1) to the whole SD vector. Not found upstream. AutoBayes applies what the table says (D187).

Eleven more were found and are waiting for a reproduction script before they are logged. The ones that give a wrong value:

- **the inverse Gaussian's right tail**, used by right censoring and `trunc(lb)`, has one sign flipped: at $\mu = 2$, $\lambda = 3$, $y = 1.5$ its log survival is -0.259 instead of -0.685;
- `cens()` **with** `trunc()`: a censored row gets the untruncated tail minus the truncation mass, which can exceed probability 1 (log $+0.209$ where the exact value is 0);
- `trunc(lb = 0)` **in zero-inflated and hurdle families** mishandles the zero atom; for the continuous ones the density integrates to 1.43;
- `mo()`, `me()` **or** `mi()` **times a factor of three or more levels**: an R error with the factor's main effect, and without it every level's term multiplied by the first level's indicator (log density off by 1 to 872);
- `cs()` **in the cumulative family**: thresholds not kept ordered, so the log density is NaN at 568 of 800 points and `posterior_epred` gives negative probabilities, while brms only calls it "experimental";
- **a multivariate model under rescor with one response unweighted**: the joint density weighted by the other response's weights, or Stan code that does not compile;
- `gp(..., iso = FALSE)` **with a Matern kernel**: the exact and approximate processes use different kernels.

The other four are smaller: a mixture's truncation normalises each component separately (a proper density, but not the truncated mixture); the default length-scale prior is rounded down in its sixth decimal at some distances; with `scale = FALSE` its fallback prior is not rescaled; and an error message reads "sameweights". The G07 probe found the first three of the wrong-value list by reading brms's generated code, and I checked their numbers by hand; they are not yet confirmed with compiled Stan. The drafts in `upstream-drafts/` have U54 added.

5 Does AutoBayes read brms's syntax?

Of the 248 corpus formulas, 193 now build brms's data (134 at the Tier F boundary), 54 are refused with the construct named (113), and the one documented difference remains (the Gamma family's default link, D9). Nothing is misread. The refusals left are spread out: addition terms on families that do not take them yet (11), family–response pairs (9), population-level terms such as `s()` and `t2()` (8), special terms (6) and `bf()` options (6). Most are G07's cells or splines.

G07, the family–context matrix. A read-only probe crossed every family (and link) with every context: `cens()` of each kind, `trunc()`, `weights()`, `se()`/`rate()`, `offset()`, a distributional formula on each parameter, and group terms in mu and in those formulas. Of 1,667 cells both libraries build 369 and both refuse 459. In 839 brms builds and AutoBayes refuses: 8 refusals kept on purpose, 95 where brms's model looks wrong (the first three items above among them), and 736 gaps worth closing. No cell has AutoBayes building what brms refuses. The probe proposes about seventeen features and 140 frozen cases (Section 9). It also found that one generic refusal, behind 476 of the 839, cites no divergence row.

6 What combining the features found

- **Fields that look set.** G05b added a field to group terms with a default of `False`; G06's check for group terms in distributional formulas counted it as set and refused `sigma ~ z + (1 | g)`. It now compares each field with its default.
- **A latent adapter bug.** G09 found that the oracle adapter's GP branch overwrote a variable, so any case with a GP and a group term failed with a KeyError. No earlier case had both.
- **Merging onto a moved `main`.** G08, G09 and G10 ran in parallel; G09 and G10 each finished after another had merged and re-integrated onto the new `main`. Where an agent's full suite ran before its last rebase or before a one-line fix, I re-ran the affected checks on the final tree: the full oracle suite and the GP, multivariate, prior and syntax tests for G09 (3,554 passed), G10's tests and cases (309 passed), and G08's and G11's changed files.
- **Lost state.** The operating system cleared the scratch directory, taking the agents' briefs with it; they are rebuilt from the session's transcript into a durable notes folder. A network outage stopped two agents after their gates had passed; their records were folded in and nothing was rerun.

7 How the claims were checked, and their limits

- **Per feature.** The gate tool ran G1–G6 on the tree that merged, diff coverage was 100% on every feature, and every agent ran the full non-sampling suite. Main's case models were snapshotted before each feature and compared after.
- **Where brms's number is not the target.** Six cases keep per-gate expected failures, each where brms is wrong and AutoBayes is checked independently instead: `gauss_gr_cov_slope_signed` (G2, G3; D185), `gauss_sigma_shared_id` (G2; D187), `gauss_mo_colon_factor` (G2–G4; D197, a scipy computation at all eight points), `gauss_gp_by_factor` (G6 only; brms's rounding, D67); and two accepted against 50-digit references, `negbinomial_truncated` (D121) and `gauss_loo_compare_x_stacking` (D126). `cumulative_cs` stays gated because AutoBayes refuses it (D89).
- **Hosted CI** now runs (the billing block is gone), but each run was cancelled by the ledger commit pushed right after its merge, and the one left to run, on `main` after G10, hit the job's 40-minute limit: the fast job also runs the sampling tests, and the suite has outgrown it. Merges rested on local verification, as you decided on 15 October.
- **Limits.** Apart from the benchmark, nothing samples. No Tier G construct has been fitted outside its frozen points, and G11's large models have only been built and evaluated, never fitted. The pilot is one seed at the smallest size.

8 Process and cost

Agent tokens reported, in millions: features G06 0.94, G05b 0.92, G05a 0.84, G10 0.81 (with its re-integration), G11 0.72, G02 0.71, G03 0.68, G04 0.68, G09 0.57, G08 0.54, G01 0.47; fixtures about 1.5 (three agents); the G07 probe 0.35, the scale investigation 0.30, the benchmark scripts 0.24 and the prediction fix 0.25. About 10.5M in all, against Tier F's 14.3M. The pace was set by the machine: with three agents, swap stayed near full and a full suite took up to two hours (load average up to 112). Weekly usage was 32% at the boundary, well under your 90% stop.

9 Decisions

1. **G07's scope.** I recommend a first pass of six features, about 46 cases: `weights()` for eleven families; `trunc()` for lognormal, Weibull and Gamma; censoring and truncation for the count families; group terms in the distributional formulas already offered; and the Student-t, Beta, ex-Gaussian and inverse-Gaussian tails. The hard ones (mixture contexts, Wiener `ndt` and `bias`, skew-normal and von Mises tails) would wait. The alternatives are all seventeen (about three times the agent time) or deferring G07.
2. **The full benchmark grid at 100k and 1M rows.** (A) all of it: 300 tasks, about 3,200 core-hours; (B), which I recommend: S1–S4, three seeds, five configurations (cmdstanr without threads, with 4 threads, 4 static, JK's setting, and AutoBayes on 4 cores), 120 tasks, about 1,300 core-hours. With it: lift the 1,000-group cap at 1M rows to $J = N/100$ (new data in a new folder), and fix or drop S5.
3. **Posting to brms.** The drafts are ready and include U54. U53 is already fixed upstream. I would still post the maintainer note alone first, after the eleven pending items have scripts.
4. **A new identity digest** to remove the 197 s of hashing at $10^6 \times 10^4$. It changes every group model's digest, so only with your yes.
5. **Hosted CI.** The fast job (40-minute limit) runs every test but the slow ones, sampling tests included, and no longer fits. I suggest running `-m "not slow and not sampling"` there, as the loop does locally, and moving the sampling tests to the slow workflow; or raising the limit. It is a change to the workflow file, so I have not made it.
6. **Splines** still wait on the licensing question (B04).
7. **"Gibbs gamma conditioning"** on your to-do list: which project is it?